Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

BAGONG PILIPINAS

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Requirements-Driven Data Modeling and Normalization for Your Semester System

ERD, Normalized Relational Schema, Data Dictionary, and Traceability

Course	IT 108 - Advanced Database Management System
Activity	Laboratory Activity No. 4
Output	Project ERD, normalized PostgreSQL schema, data dictionary, requirements-to-data traceability, and group presentation
Maximum Score	80 points

Student / Project Information

Block: _B2_____	Date: _10–3-2026_____
System Title: PUBLIC UTILITY VEHICLE PASSENGER COUNTING USING YOLO-BASED COMPUTER VISION	Instructor: Jean Y. Villasin, MIT
Members: RITZY BERNA CATAP, SHANGER LOR, JEZER AYUBAN	

1. Activity Alignment and Purpose

Topic Learning Outcome: Translate finalized system requirements and user workflows of the developed semester system into a normalized relational model and PostgreSQL schema.

Course Outcome Alignment: CO2 - Translate finalized system requirements and user workflows of the developed semester system into a normalized relational model and PostgreSQL schema.

Syllabus Assessment: Project ERD and normalized schema, data dictionary, and requirements-to-data traceability.

Syllabus Laboratory Output: Project ERD and normalized PostgreSQL schema, completed data dictionary, and requirements-to-data traceability documentation.

Purpose of the Activity

This activity converts the team's finalized system requirements, use cases/user stories, and user workflows into a defensible database design. The group will identify the data that must persist, model entities and relationships, analyze functional dependencies, normalize the model, and translate the final ERD into a PostgreSQL-ready relational schema. The resulting database model will serve as the structural foundation for API development, advanced querying, testing, and later system integration.

Important distinction: A polished ERD is not sufficient if its entities, attributes, relationships, keys, and constraints cannot be traced to actual system requirements. Every major table must exist for a clear business or workflow reason, and every normalization decision must be explainable.

2. Intended Learning Objectives

•Derive persistent data requirements from finalized system requirements, use cases, user stories, and workflows. •Identify appropriate entities, attributes, candidate keys, relationships, cardinalities, and optionalities for the project. •Create a complete project ERD and map it to relational tables, primary keys, foreign keys, and associative entities. •Analyze functional dependencies and normalize core relations through 1NF, 2NF, 3NF, and BCNF evaluation. •Evaluate denormalization trade-offs and justify whether the final design should remain normalized or selectively denormalize a specific case.

•Apply consistent PostgreSQL naming, schema, data type, key, and integrity conventions.

•Complete a data dictionary and requirements-to-data traceability matrix that connect project features to final tables and attributes.

•Explain and defend modeling decisions during peer review and oral presentation.

3. Activity Specifications

Item	Requirement
Mode	Existing semester-project group; all members must participate in requirements analysis, ERD refinement, normalization, and schema review.
Suggested Time	One laboratory session, approximately 3 hours, plus peer review and the scheduled group presentation/oral defense.
Primary Output	One finalized project ERD and normalized relational/PostgreSQL schema based on the team's actual semester system.
Supporting Output	Functional-dependency and normalization evidence, data dictionary, requirements-to-data traceability sheet, peer-review record, and schema documentation.
Recommended Tools	diagrams.net/Draw.io, dbdiagram.io, Lucidchart, Figma, pgAdmin 4, DBeaver, PostgreSQL, or equivalent modeling/database tools; GitHub for version control.
Presentation Format	5-7 minutes per group followed by 2-3 minutes of questions. Every member must explain at least one modeling or normalization decision.

4. Pre-Laboratory Requirements

•Finalized or instructor-approved system title, problem statement, primary users, and major system features. •Current use cases, user stories, workflow descriptions, or feature specifications from the semester system, including outputs expected by each user role.

•Draft ERD, table list, or existing PostgreSQL schema if the group has already started database design. •A working PostgreSQL environment or database client for checking data types, key definitions, and constraints. •A list of important business rules such as uniqueness, required fields, status values, ownership rules, approval rules, and parent-child relationships.

•A project repository/shared folder for the ERD source file, schema SQL, data dictionary, traceability matrix, and review evidence.

5. Part A - Requirements-to-Data Inventory

Before drawing or revising the ERD, identify what data each important user workflow creates, reads, updates, or deletes. Use requirement IDs or feature IDs already used by the team whenever available. Do not invent tables first and then search for a requirement to justify them.

	Use Case / User Story /		Data Created / Read /		Key Business Rule
R01	Core user/account workflow	Admin / Operator	User account information is created, read, and updated.	users_tbl	Each user must have a user account and assigned role.

R02	Primary passenger-counting workflow	Operator	Passenger-counting records, image information, passenger count, and processing status are created and read.	counting_records_tbl	Each counting record represents a passenger-counting process and has a processing status.
R03	Counting record details workflow	Admin / Operator	Counting record and associated user and PUV information are read.	`counting_records_tbl`, `users_tbl`, `puvs_tbl`	A counting record may be associated with the user who processed it and the PUV being counted.
R04	PUV reference/master-data feature	Admin	PUV information is created, read, and updated.	puvs_tbl	Each registered PUV must have a unique PUV identifier.
R05	Counting processing status/history workflow	Admin / Operator	Processing status and processing date/time are created, read, and updated.	counting_records_tbl	A counting record may have a status such as pending, processing, completed, or failed.
R06	YOLO detection/image-processing feature	Admin / Operator	YOLO detection results, object class, confidence, and bounding-box coordinates are created and read.	`detection_tbl`, `counting_records_tbl`	Each detection result must be associated with its corresponding counting record.
R07	Dashboard/reporting requirement	Admin / Operator	Passenger counts, processed records, averages, totals, and PUV information are read and summarized.	`counting_records_tbl`, `puvs_tbl`	Dashboard and report values should be derived from stored passenger-counting records.
R08	PUV passenger-counting analytics	Admin	Passenger counts, PUV information, and detection results are read and analyzed.	`puvs_tbl`, `counting_records_tbl`, `detection_tbl`	Analytics should use completed passenger-counting records and available detection data.

Requirements-to-Data Rule

Every major persistent entity in the final ERD must be traceable to at least one requirement or workflow. Temporary UI state, calculated display values, and external data should not automatically become database tables unless the system has a clear need to persist them.

6. Part B - Entity Identification, Relationship Analysis, and ERD Refinement

A. Entity Inventory

List the core entities derived from Part A. Use project-specific names. Separate people/roles from accounts when the requirements require different concepts, and distinguish transaction data from reference/master data.

			Candidate Primary		Source Req. ID(s)
1	users_tbl	Stores system user/account information and roles.	user_id	`full_name`, `email`, `password_hash`, `role`	R01, R03
2	puvs_tbl	Stores registered PUV information.	puv_id	`plate_number`, `puv_type`	R04, R07, R08
3	counting_records_tbl	Stores passenger-counting records, image information, passenger count, and processing information.	count_id	`user_id`, `puv_id`, `image_path`, `passenger_count`, `status`, `processed_at`	R02, R03, R05, R07, R08
4	detection_tbl	Stores individual YOLO detection results associated with counting records.	detection_id	`count_id`, `object_class`, `confidence`, `x1`, `y1`, `x2`, `y2`	R06, R08
5					
6					
7					
8					

B. Relationship and Cardinality Worksheet

					FK / Junction Table
1	users_tbl	processes	counting_records_tbl	1 : 0..many	counting_records_tbl.user_id
2	puvs_tbl	has	counting_records_tbl	1 : 0..many	counting_records_tbl.puv_id
3	counting_records_tbl	contains	detection_tbl	1 : 0..many	detection_tbl.count_id
4					
5					
6					
7					
8					

C. Final ERD Requirements

No.	ERD Element	What Must Be Shown
1	System identity	Official project title and ERD version/date.
2	Entities	All persistent project entities using actual project names.
3	Keys	Primary keys and foreign keys clearly marked.

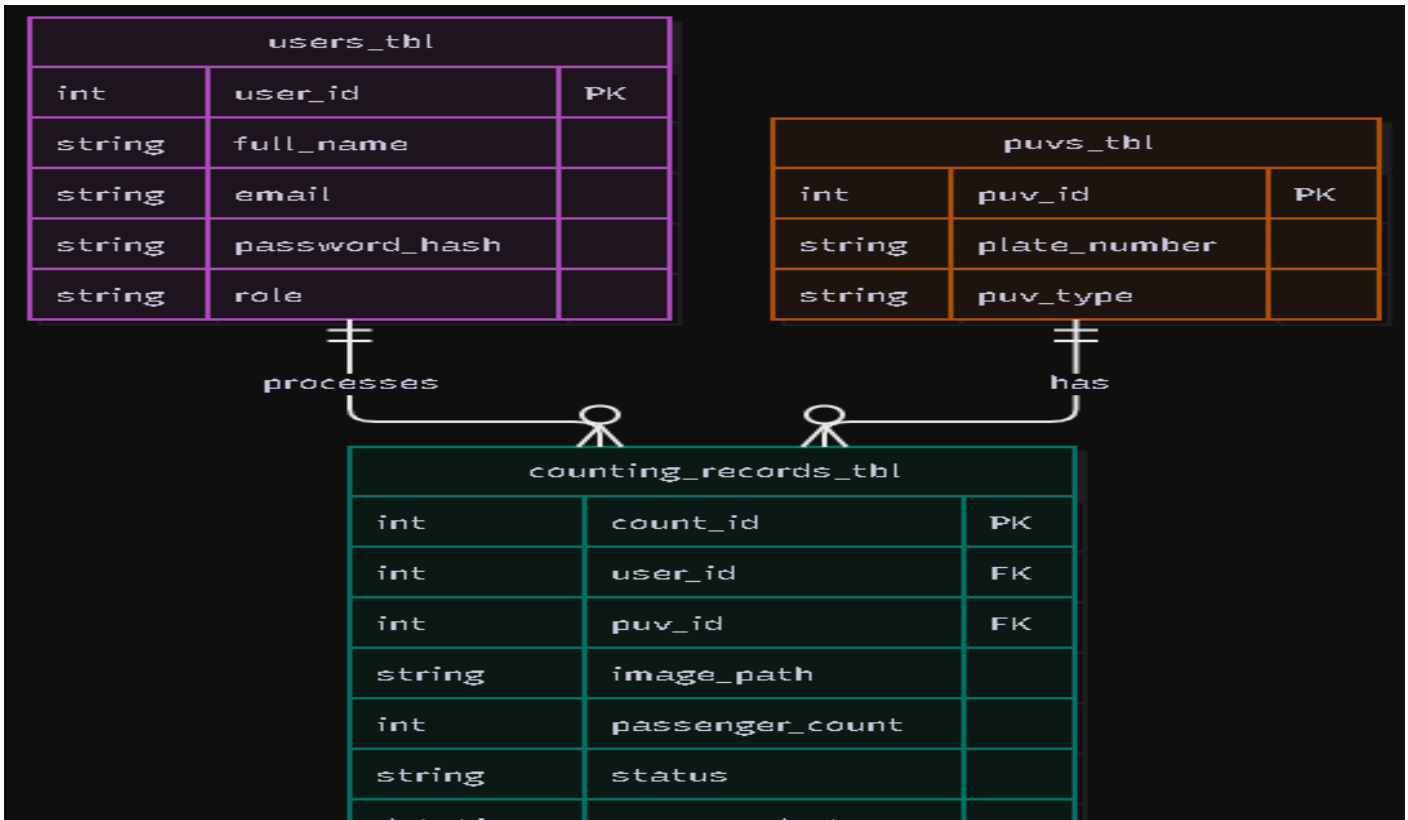
4	Relationships	Relationship lines with correct one-to-one, one-to-many, or many-to-many meaning.
5	Cardinality / optionality	Minimum and maximum participation shown or documented consistently.
6	Many-to-many resolution	Associative/junction entities used when required.
7	Attributes	Important attributes shown at a readable level; avoid crowding with irrelevant implementation detail.
8	Integrity rules	Unique, required, constrained, and ownership relationships represented or documented.
9	Legend / notation	Crow's-foot or another accepted notation applied consistently and explained.

Minimum ERD Rule

ERD Element	What Must Be Shown

No.

The final ERD must support every major workflow listed in Part A and must not contain unresolved many-to-many relationships. A reviewer should be able to follow a user workflow and identify where its records are stored, how those records relate, and which keys preserve integrity.



7. Part C - Functional Dependencies and Normalization Workshop

Normalization must be based on dependency reasoning, not only on visual preference. For each selected workflow or relation, identify the determinant, dependent attributes, candidate key, and the anomaly or redundancy that would occur if the relation remained poorly structured.

A. Functional Dependency Worksheet

					Action
1	PUV Information	user_id	user_id → full_name, email, password_hash, role	Full dependency on primary key	Keep as `users_tbl`
2	PUV Information	puv_id	puv_id → plate_number, puv_type	Full dependency on primary key	Keep as `puvs_tbl`
3	Passenger Counting Record	count_id	count_id → user_id, puv_id, image_path, passenger_count, status, processed_at	Full dependency on primary key	Keep as `counting_records_tbl`
4	YOLO Detection Result	detection_id	detection_id → count_id, object_class, confidence, x1, y1, x2, y2	Full dependency on primary key	Keep as `detection_tbl`
5	Counting Record–User Relationship	count_id	count_id → user_id	Identifies the user associated with the counting record	Keep `user_id` as FK in `counting_records_tbl`

6	Counting Record–PUV Relationship	count_id	count_id → puv_id	Identifies the PUV associated with the counting record	Keep `puv_id` as FK in `counting_records_tbl`

B. Normalization Sequence

Stage	Required Analysis / Evidence
1NF	Ensure each field is atomic, remove repeating groups, and identify a valid key.
2NF	If a composite key exists, remove attributes that depend on only part of that key.
3NF	Remove transitive dependencies so non-key attributes depend on the key, the whole key, and nothing but the key.
BCNF	Evaluate whether every determinant is a candidate key. Decompose a violation when doing so preserves the intended dependencies and is appropriate for the project.
Final check	Confirm keys, foreign keys, relationship cardinalities, constraints, and requirement traceability after decomposition.

C. Normalization Evidence Sheet

					BCNF Evaluation / Final Relations
N01	User account data	Attributes are atomic and each account is identified by `user_id`.	Single-column key, so there is no partial dependency.	User attributes depend only on `user_id`.	No documented BCNF violation. Final: `users_tbl`.
N02	PUV information	PUV attributes are atomic and identified by `puv_id`.	Single-column key, so there is no partial dependency.	PUV attributes depend only on `puv_id`.	No documented BCNF violation. Final: `puvs_tbl`.
N03	Passenger counting data	Each record contains atomic attributes and is identified by `count_id`.	Non-key attributes depend on the complete single-column key.	User and PUV information are referenced through foreign keys instead of duplicated descriptive information.	No documented BCNF violation. Final: `counting_records_tbl`.
N04	YOLO detection data	Each detection is an individual record	Single-column key means there is no	Detection attributes depend on	No documented BCNF violation. Final: `detection_tbl`.

		identified by `detection_id`.	partial dependency.	`detection_id`, while the related counting record is referenced through `count_id`.	

D. Before-and-After Decomposition

Before Normalization .

COUNTING_DATA(
count_id,
user_id,
full_name,
email,
password_hash,
role,
puv_id,
plate_number,
puv_type,
image_path,
passenger_count,
status,
processed_at,
detection_id,
object_class,
confidence,
x1,
y1,
x2,
y2
)

After Normalization

users_tbl(
user_id,
full_name,
email,
password_hash,
role
)

puvs_tbl(
puv_id,
plate_number,
puv_type
)

counting_records_tbl(
    count_id,
    user_id,
    puv_id,
    image_path,
    passenger_count,
    status,
    processed_at
)

detection_tbl(
    detection_id,
    count_id,
    object_class,
    confidence,
    x1,
    y1,
    x2,
    y2
)

Explanation:
The combined data is separated into four relations to reduce unnecessary duplication and keep user, PUV, passenger-counting, and YOLO detection information in their appropriate relations. Primary keys identify each record, while foreign keys maintain the relationships between the tables.

Minimum Normalization Rule

All core transactional relations must be at least in Third Normal Form (3NF), and every final relation must be evaluated for BCNF. At least one case must show a clear before-and-after decomposition. If the team's existing design is already normalized, reconstruct a requirement-level combined relation from one workflow and document why the final decomposed design is superior.

8. Reference Example - Do Not Copy as Your Final Output

The generic example below illustrates the reasoning expected. Replace it with your own project entities, dependencies, business rules, and names.

Example: Unnormalized Service Request Data

Item	Example Evidence

Item	Example Evidence
Combined relation	REQUEST_DATA(request_id, request_title, category_id, category_name, created_by, creator_name, creator_email, created_at)
Candidate key	request_id
Functional	request_id -> request_title, category_id, created_by, created_at; category_id -> category_name; created_by

dependencies	-> creator_name, creator_email
Problem	Category and user details repeat across many requests. Updating a category name or user email in duplicated request rows can create inconsistencies.
3NF decomposition	service_request(request_id, request_title, category_id, created_by, created_at); category(category_id, category_name); app_user(user_id, full_name, email)
Integrity mapping	service_request.category_id -> category.category_id; service_request.created_by -> app_user.user_id

Why the Decomposition Is Better

Each fact is stored once in the relation where it depends on the key. Category details belong to category, user details belong to app_user, and request-specific facts remain in service_request. The final structure reduces update anomalies while preserving relationships through foreign keys.

9. Denormalization Trade-Off Analysis

Normalization is the default. Denormalization is allowed only when the team identifies a concrete performance, reporting, offline, or integration need and can explain how consistency will be protected.

				Decision & Consistency Safeguard
1	Store PUV information such as `plate_number` and `puv_type` directly in `counting_records_tbl` instead of referencing `puvs_tbl`.	Could reduce joins when displaying counting records and reports.	Repeated PUV information may become inconsistent if the plate number or PUV type is changed.	**Do not denormalize.** Keep PUV information in `puvs_tbl` and use `puv_id` as the foreign key in `counting_records_tbl`.
2	Store user information such as `full_name` and `email` directly in `counting_records_tbl` instead of referencing `users_tbl`.	Could make some reports easier to retrieve without joining the user table.	User information would be duplicated across counting records and could become inconsistent when account information changes.	**Do not denormalize.** Keep user information in `users_tbl` and use `user_id` as the foreign key in `counting_records_tbl`.

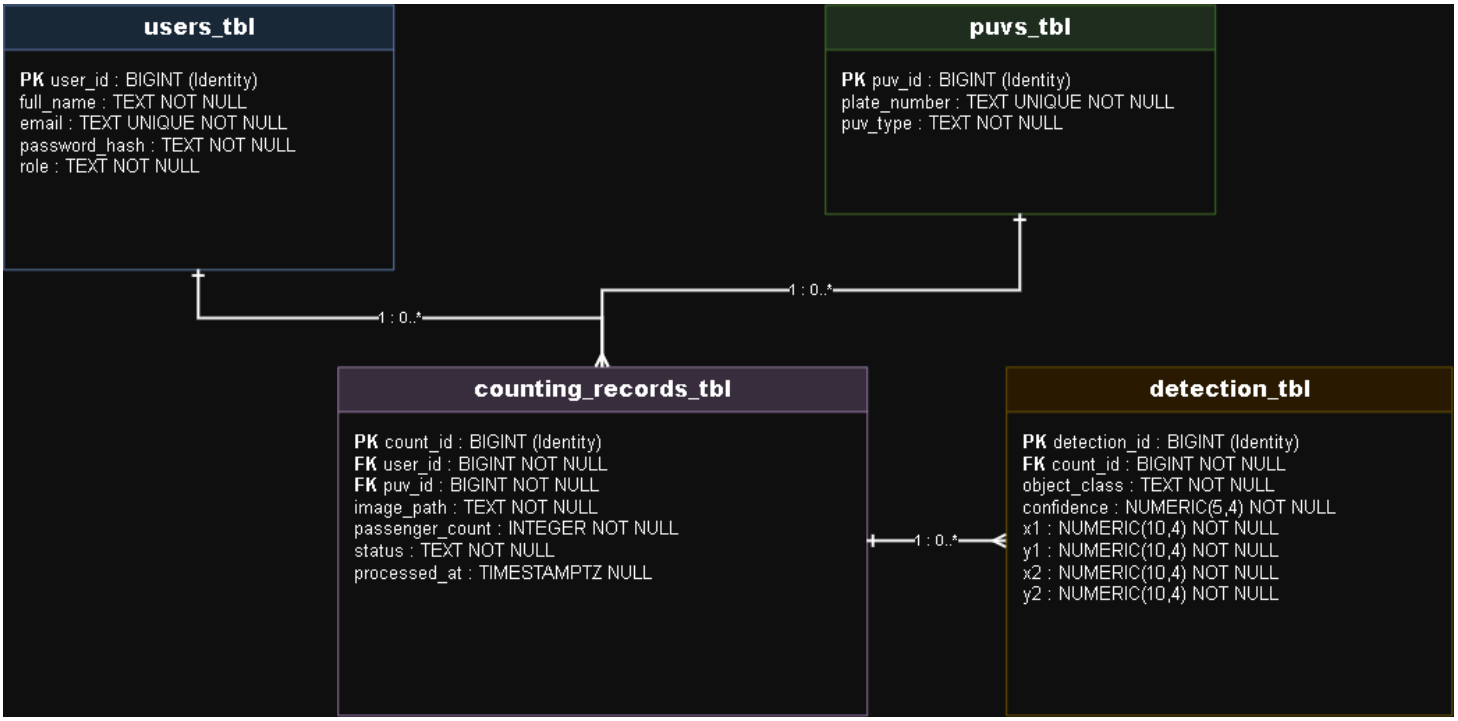
10. Recommended PostgreSQL Modeling Conventions

	Recommended Convention
Naming	Use one consistent convention; lowercase snake_case is recommended. Avoid quoted mixed-case identifiers and PostgreSQL reserved words.
Primary keys	Use a stable identifier appropriate to the entity; document whether it is generated, natural, UUID, identity, or another justified key.
Foreign keys	Name and define FKs consistently; ensure the referenced key is correct and define ON DELETE / ON UPDATE behavior intentionally.
Required data	Use NOT NULL when the business rule requires a value; do not rely only on application-side validation.
Uniqueness	Use UNIQUE for business identifiers that must not repeat, such as approved reference codes or account emails where applicable.
Domain rules	Use CHECK constraints for bounded values, valid ranges, or state rules when they can be enforced safely in PostgreSQL.
Data types	Choose types that match the data meaning: date/timestamptz for time, numeric for exact decimal values, boolean for true/false, text/varchar only when appropriate.
Audit/history	Store created/updated timestamps and history records only when the workflow requires them; do not overwrite historical facts that must be traceable.
Schema ownership	Place project objects in a documented PostgreSQL schema when appropriate and keep

	environment-specific names out of the logical model.

11. Part D - Construct the Final Project ERD and Relational PostgreSQL Schema

Refine the ERD using the normalization results, then translate every final entity into a relational table definition. The ERD and PostgreSQL schema must agree on table names, key relationships, and major constraints.



No.	Schema Element	What Must Be Shown
1	Table identity	`users_tbl` — R01 Core user/account workflow `puvs_tbl` — R04 PUV reference/master-data feature `counting_records_tbl` — R02 Primary passenger-counting workflow `detection_tbl` — R06 YOLO detection/image-processing feature

No.	Schema Element	What Must Be Shown

2	Primary key	`users_tbl` — `user_id`
		`puvs_tbl` — `puv_id`
		`counting_records_tbl` — `count_id`
		`detection_tbl` — `detection_id`

3	Foreign keys	counting_records_tbl.user_id → users_tbl.user_id counting_records_tbl.puv_id → puvs_tbl.puv_id detection_tbl.count_id → counting_records_tbl.count_id
4	Data types	To be finalized based on the PostgreSQL schema.
5	Nullability	Required identifiers and relationship fields should be NOT NULL where required by the business rules. Optional fields may allow NULL.
6	Unique rules	users_tbl.email — UNIQUE puvs_tbl.plate_number — UNIQUE
7	Check/domain rules	counting_records_tbl.status should allow only pending, processing, completed, or failed.
8	Referential actions	To be finalized based on the PostgreSQL schema.
9	Normalization status	All final relations are at least in 3NF. BCNF was evaluated, and no documented BCNF violation was identified.
10	Traceability	users_tbl — R01, R03 puvs_tbl — R04, R07, R08 counting_records_tbl — R02, R03, R05, R07, R08 detection_tbl — R06, R08

Relational Mapping Worksheet

					Requirement ID(s)
1	users_tbl	user_id	None	full_name, email, password_hash, role	3NF / BCNF evaluated
2	puvs_tbl	puv_id	None	plate_number, puv_type	3NF / BCNF evaluated
3	counting_records_tbl	count_id	user_id → users_tbl.user_id; puv_id → puvs_tbl.puv_id	image_path, passenger_count, status, processed_at	3NF / BCNF evaluated
4	detection_tbl	detection_id	count_id → counting_records_tbl.count_id	object_class, confidence, x1, y1, x2, y2	3NF / BCNF evaluated
5					
6					
7					

8					

PostgreSQL Schema Verification

Submit a schema SQL file or equivalent PostgreSQL definition that can create the final tables and constraints in dependency-safe order. The script must match the ERD. The instructor may ask the group to explain a data type, key, foreign-key action, uniqueness rule, or normalization decision.

Generic DDL Skeleton - Use Your Own Names

```
CREATE SCHEMA IF NOT EXISTS project_schema;
CREATE TABLE project_schema.entity_name (
 entity_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 related_id bigint NOT NULL REFERENCES project_schema.related_entity(related_id),
 business_code text NOT NULL UNIQUE,
 status text NOT NULL CHECK (status IN ('ACTIVE', 'INACTIVE')),
 created_at timestamptz NOT NULL DEFAULT now()
);
```

12. Part E - Data Dictionary and Requirements-to-Data Traceability

A. Data Dictionary

The final data dictionary must document every table and column in the submitted schema. The worksheet below shows the required fields; continue the table in a separate document or spreadsheet if needed.

							Source Req.
users_tbl	user_id	BIGINT	No	PK, Identity	Auto-generated	Unique identifier for each system user.	
users_tbl	full_name	TEXT	No	NOT NULL	—	Full name of the system user.	

							Source Req.
users_tbl	email	TEXT	No	UNIQUE, NOT NULL	—	Email used to identify the user's account.	R01
users_tbl	password_hash	TEXT	No	NOT NULL	—	Stores the hashed password for account authentication.	R01
users_tbl	role	TEXT	No	NOT NULL	—	Identifies the user's system role, such as Admin or Operator.	R01
puvs_tbl	puv_id	BIGINT	No	PK, Identity	Auto-generated	Unique identifier for each registered PUV.	R04
puvs_tbl	plate_number	TEXT	No	UNIQUE, NOT NULL	—	Unique plate number of the registered PUV.	R04
puvs_tbl	puv_type	TEXT	No	NOT NULL	—	Identifies the type of PUV, such as bus, van, or multicab.	R04
counting_records_tbl	count_id	BIGINT	No	PK, Identity	Auto-generated	Unique identifier for each passenger-counting record.	R02
counting_records_tbl	user_id	BIGINT	No	FK, NOT NULL	—	Identifies the user who processed the counting record.	R03
counting_records_tbl	puv_id	BIGINT	No	FK, NOT NULL	—	Identifies the PUV associated with the counting record.	R03

counting_records_tbl	image_path	TEXT	No	NOT NULL	—	Stores the path or location of the image used for passenger counting.	R02, R06
counting_records_tbl	passenger_count	INTEGER	No	CHECK ≥ 0, NOT NULL	—	Stores the number of passengers detected in the counting process.	R02, R07
counting_records_tbl	status	TEXT	No	CHECK	—	Processing status: pending, processing, completed, or failed.	R05
counting_records_tbl	processed_at	TIMESTAMPTZ	YES	—	—	Records the date and time when the counting process was completed or processed.	R05
detection_tbl	detection_id	BIGINT	No	PK, Identity	Auto-generated	Unique identifier for each YOLO detection result.	R06
detection_tbl	count_id	BIGINT	No	FK, NOT NULL	—	Identifies the counting record associated with the detection.	R06
detection_tbl	object_class	TEXT	No	NOT NULL	—	Stores the detected object class produced by YOLO.	R06
detection_tbl	confidence	NUMERIC(5,4)	No	CHECK 0–1, NOT NULL	—	Stores YOLO's confidence score for the detection.	R06
detection_tbl	x1	NUMERIC(10,4)	No	NOT NULL	—	X-coordinate of the first bounding-box point.	R06
detection_tbl	y1	NUMERIC(10,4)	No	NOT NULL	—	Y-coordinate of the first bounding-box point.	R06
detection_tbl	x2	NUMERIC(10,4)	No	NOT NULL	—	X-coordinate of the second bounding-box point.	R06
detection_tbl	y2	NUMERIC(10,4)	No	NOT NULL	—	Y-coordinate of the second bounding-box point.	R06

Data Dictionary Rule

A data dictionary is not only a list of data types. Each column must have a clear business meaning, key/constraint information, nullability/default behavior, and a source requirement or workflow. Avoid vague descriptions such as “stores data.”

B. Requirements-to-Data Traceability Matrix

			CRUD / Data			Evidence / Status
1	R01 – Core user/account workflow	Admin / Operator	Create, Read, Update	users_tbl	`user_id` PK, `email` UNIQUE, required account fields	Implemented and verified
2	R02 – Primary passenger-counting workflow	Operator	Create, Read	counting_records_tbl	`count_id` PK, `passenger_count`, `status`, `image_path`	Implemented and verified
3	R03 – Counting record details workflow	Admin / Operator	Read	`counting_records_tbl`, `users_tbl`, `puvs_tbl`	FKs `user_id` and `puv_id` preserve relationships	Implemented and verified
4	R04 – PUV reference/master-data feature	Admin	Create, Read, Update	puvs_tbl	`puv_id` PK, `plate_number` UNIQUE	Implemented and verified
5	R05 – Counting processing status/history workflow	Admin / Operator	Create, Read, Update	counting_records_tbl	`status` CHECK, `processed_at`	Implemented and verified
6	R06 – YOLO	Admin / Operator	Create, Read	`detection_tbl`,	`count_id` FK, confidence CHECK	Implemented

	detection/image-processing feature			counting_records_tbl	0–1	and verified
7	R07 – Dashboard/reporting requirement	Admin / Operator	Read	counting_records_tbl, puvs_tbl	Passenger count and PUV relationships support summaries	Implemented and verified
8	R08 – PUV passenger-counting analytics	Admin	Read	puvs_tbl, counting_records_tbl, detection_tbl	Related PUV, counting, and detection data support analytics	Implemented and verified

Traceability Guidance

- Map each major feature or user workflow to the specific table(s) that persist or retrieve its data.
- Use CRUD labels (Create, Read, Update, Delete) where helpful to show how the workflow interacts with the schema.
- Identify the key constraint that protects the requirement, such as UNIQUE, NOT NULL, FK ownership, CHECK, or a required relationship.
- If a feature uses no persistent database data, document that decision instead of inventing a table.
- The traceability matrix, ERD, data dictionary, and schema SQL must use the same names and must not contradict one another.

13. Laboratory Procedure

Phase 1 - Review finalized requirements (20 minutes)

Confirm the approved users, use cases/user stories, major workflows, outputs, and business rules. Complete Part A.

Phase 2 - Conduct the design critique (25 minutes)

Review the draft ERD. Identify missing entities, duplicate concepts, unresolved many-to-many relationships, weak keys, unclear optionality, and requirements with no data destination.

Phase 3 - Run the normalization workshop (45 minutes)

Complete the functional-dependency worksheet and normalization evidence for selected core workflows. Normalize to at least 3NF and evaluate BCNF.

Phase 4 - Refine the ERD and relational mapping (35 minutes)

Update entities, keys, foreign keys, cardinalities, associative entities, names, and integrity rules. Complete the relational mapping worksheet.

Phase 5 - Build the PostgreSQL-ready schema and dictionary (30 minutes)

Define data types, nullability, PK/FK/UNIQUE/CHECK constraints, defaults, and referential actions. Complete the data dictionary.

Phase 6 - Complete cross-course requirements-to-data mapping (20 minutes)

Trace finalized semester-system features/workflows to tables and constraints. Resolve any requirement that has no valid data mapping.

Phase 7 - Peer and online schema documentation review (20 minutes)

Exchange ERD/schema documentation with another group. Record issues and apply defensible revisions.

Phase 8 - Export and submit (15 minutes)

Export the final ERD, schema SQL, data dictionary, traceability matrix, normalization evidence, and peer-review record; commit them to the project repository and submit through Google Classroom

14. Peer Review / Schema Documentation Review Record

Review Item	Reviewer Comment / Evidence	Team Response / Revision
Reviewer Group / Names	**Jezer Ayuban, Ritzy Catap, Shanger Lor**	**The team received and reviewed the peer feedback.**
Strongest modeling decision	**The four-table structure clearly separates users, PUV information, counting records, and YOLO detection results.**	**The team retained the four-table structure because it provides clear relationships and avoids unnecessary data duplication.**
Issue 1 found	**The relationship between `counting_records_tbl` and `detection_tbl` should clearly represent that one counting record can have multiple detection results.**	**The team checked the ERD and verified the 1-to-many relationship between `counting_records_tbl` and `detection_tbl`.**

Issue 2 found	**The ERD, PostgreSQL schema, and data dictionary should use consistent table names, attributes, data types, and constraints.**	**The team compared the ERD, schema, and data dictionary and verified that the final versions are consistent.**
Revision applied	**The reviewers recommended checking the relationships and consistency of the database documentation.**	**The team reviewed and verified the ERD, PostgreSQL schema, data dictionary, and traceability matrix before final submission.**

15. Final Quality Checklist

	Checklist
[ ] The ERD uses the team's official system title and actual project requirements.	[ ] Every major persistent entity is traceable to at least one requirement/workflow.
[ ] Primary keys and foreign keys are clearly shown and logically correct.	[ ] Relationship cardinality and optionality match the actual business rules.
[ ] All many-to-many relationships are resolved using justified associative entities.	[ ] Functional dependencies are documented for selected core relations.
[ ] Core relations are at least in 3NF and every final relation has a BCNF evaluation.	[ ] At least one normalization case shows a clear before-and-after decomposition.
[ ] PostgreSQL data types, nullability, defaults, and integrity constraints match the ERD.	[ ] UNIQUE, CHECK, and referential actions are used only when supported by requirements.
[ ] The data dictionary documents every final table and column.	[ ] The requirements-to-data traceability matrix agrees with the ERD and schema.
[ ] Denormalization is avoided or explicitly justified with a consistency safeguard.	[ ] Naming and schema conventions are consistent across ERD, SQL, and documentation.
[ ] Peer-review findings were addressed or explicitly defended.	[ ] Every member can explain a modeling, normalization, and integrity decision without reading a script.

16. Submission Requirements

- Final project ERD as PDF or PNG in readable orientation plus the editable source file (.drawio, .dbml, .fig, .pptx, or native format).
- Normalized PostgreSQL schema SQL file containing table definitions, keys, and required integrity constraints.
- Completed normalization evidence showing functional dependencies, 1NF/2NF/3NF reasoning, and BCNF evaluation for selected core relations.
- Completed data dictionary covering every final table and column.
- Completed requirements-to-data traceability matrix linking approved system requirements/features to tables, attributes, and constraints.
- Peer-review/schema documentation record showing feedback and revisions.
- Files committed to the project GitHub repository under a folder such as database/design/ or docs/database-model/.
- Repository link and required files submitted through MasaoLMS using the naming convention: IT108_LA4_Group##_SystemName.
- Deliver a 5-7 minute group presentation using the final ERD and normalization evidence as the main visuals, followed by a 2-3 minute question-and-answer period. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The submitted model must represent the team's own semester system and design decisions. AI tools may be used to clarify database concepts, but the final entity definitions, functional dependencies, normalization steps, constraints, and traceability mappings must be reviewed, understood, and defended by the group. The instructor may ask any member to

explain a relationship, identify a dependency, normalize a relation, justify a constraint, or trace a requirement to its stored data without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. A visually attractive ERD cannot receive full credit when relationships, dependencies, constraints, or requirement mappings are incorrect.

		Points
Requirements-driven ERD correctness	Entities, attributes, keys, relationships, cardinalities, optionalities, and associative entities accurately represent the semester-system requirements and workflows.	14
Functional dependencies and normalization	The group correctly identifies dependencies and anomalies, demonstrates defensible 1NF/2NF/3NF decomposition, evaluates BCNF, and keeps the final design logically consistent.	16
Relational PostgreSQL schema and integrity	The relational mapping and schema SQL accurately implement PKs, FKs, data types, nullability, UNIQUE/CHECK rules, defaults, and referential actions that agree with the ERD.	14
Requirements-to-data traceability	Major requirements/use cases are mapped to the correct tables, data actions, critical attributes, and integrity rules with no unexplained gaps or contradictions.	10
Data dictionary completeness and accuracy	Every final table and column is documented with PostgreSQL type, nullability/default, key/constraint information, business meaning, and source requirement.	10
Denormalization analysis and modeling conventions	Any denormalization decision is justified with trade-offs and safeguards; naming, schema, and integrity conventions are consistent and PostgreSQL-appropriate.	6
Group presentation and oral defense	Presentation is clear and within time; every member participates and accurately explains the ERD, normalization evidence, schema constraints, traceability, and peer-review revisions.	10

Raw Score	____ / 80	**Equivalent**	_____
Checked by	_____	**Date**	_____

Remarks: _____

18. Post-Laboratory Reflection

Which requirement or workflow caused the largest change to your original ERD, and why?

_____

Which functional dependency or redundancy problem was most important to correct during normalization?

_____

Did any final relation remain in 3NF but require special discussion for BCNF? Explain the decision.

_____

Which integrity constraint in PostgreSQL is most important for protecting the correctness of your semester-system data?

_____

What is one case where denormalization might improve the system, and what consistency risk would it create?

_____

After peer review, what modeling issue did your group revise or defend most strongly?

_____

Which database table or relationship should be implemented and tested first when the system proceeds to API/database integration?

_____

Appendix A - Final Schema Documentation Summary

Documentation Item	Final Value

Total final tables	_____
Total junction/associative tables	_____
Core relations verified at 3NF	_____
Relations evaluated for BCNF	_____
Requirements mapped in traceability matrix	_____
Data dictionary rows / attributes documented	_____
Final ERD version / date	_____
Schema SQL file name	_____

Final reminder: The strongest database design is one where every table has a clear requirement, every relationship and dependency is defensible, and every integrity rule can be explained by the team.